

Name: Monisha Raghini Chelle

Roll No: 2520030537

Branch: B.Tech CSE

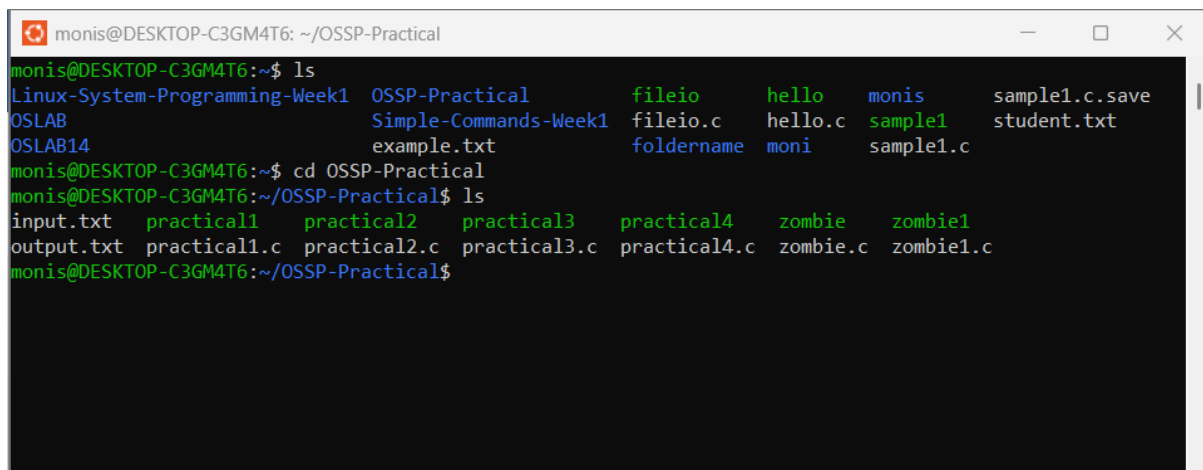
Subject: Linux System Programming Lab

OSSP_Practical5&6

OSSP (Operating Systems And Systems Programming) - 25CS2104E

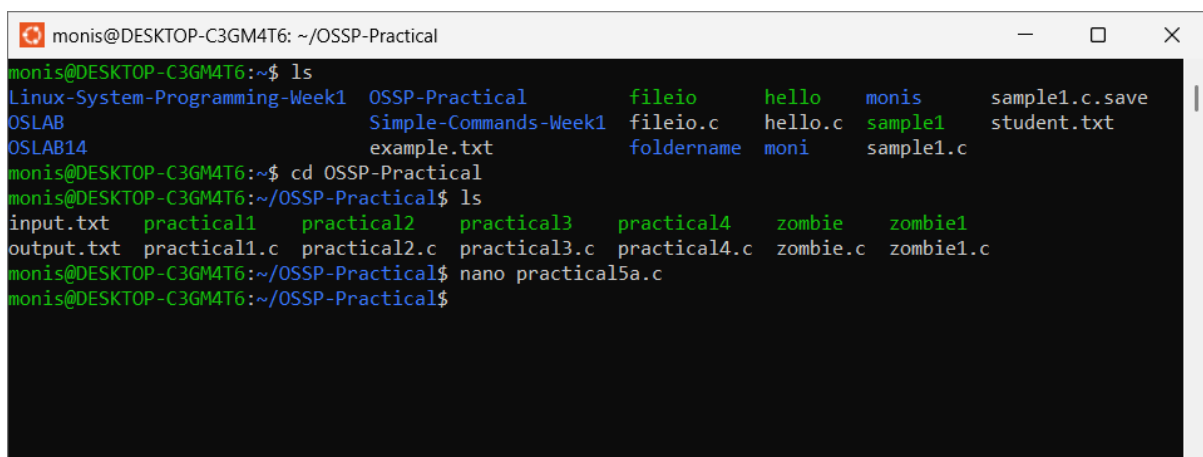
5.Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.



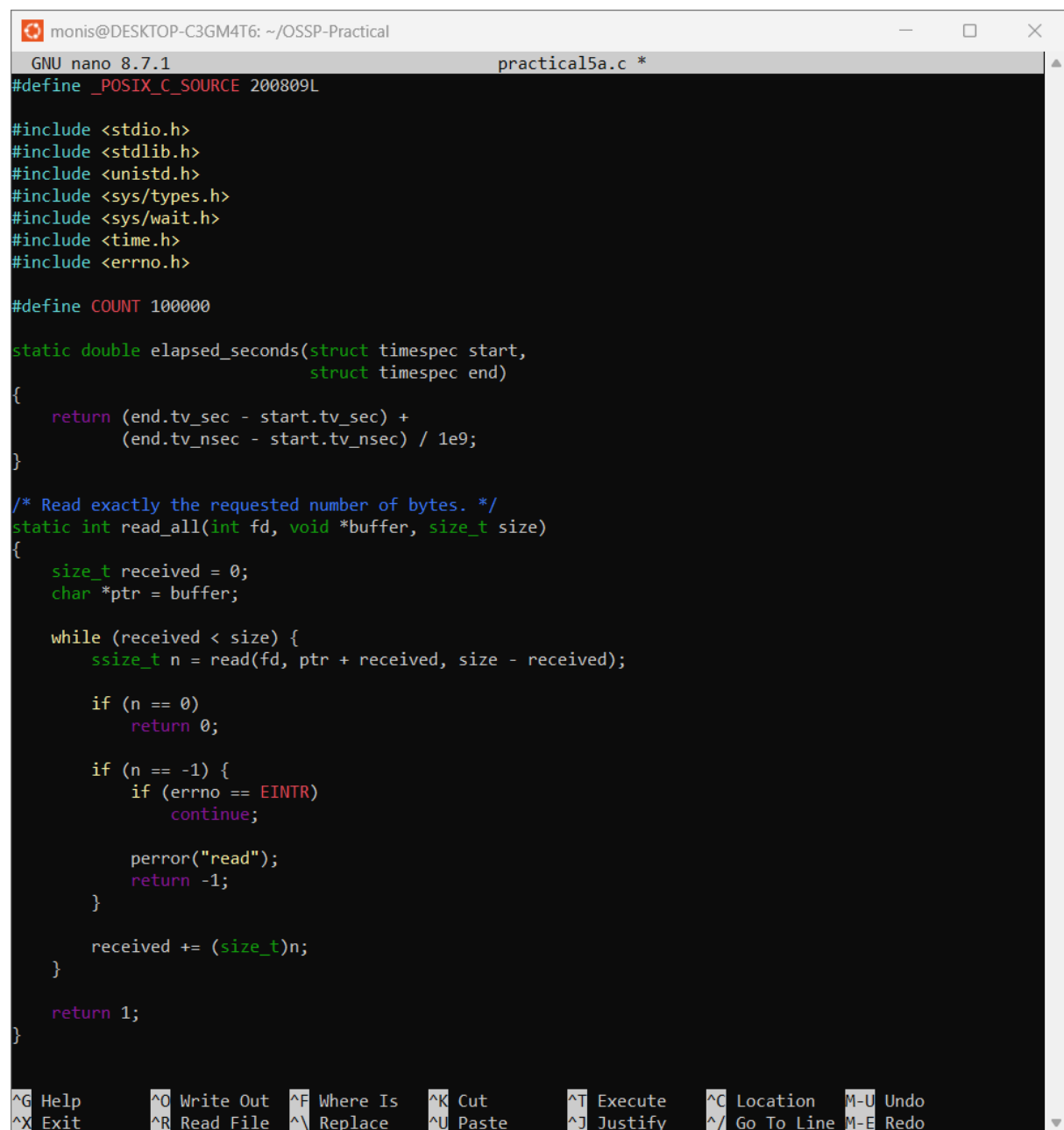
```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSSP-Practical  fileio  hello  monis  sample1.c.save
OSLAB                          Simple-Commands-Week1  fileio.c  hello.c  sample1  student.txt
OSLAB14                        example.txt      foldername  moni  sample1.c
monis@DESKTOP-C3GM4T6:~$ cd OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ls
input.txt  practical1  practical2  practical3  practical4  zombie  zombie1
output.txt practical1.c practical2.c practical3.c practical4.c zombie.c  zombie1.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 1 — Create the program



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSSP-Practical  fileio  hello  monis  sample1.c.save
OSLAB                          Simple-Commands-Week1  fileio.c  hello.c  sample1  student.txt
OSLAB14                        example.txt      foldername  moni  sample1.c
monis@DESKTOP-C3GM4T6:~$ cd OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ls
input.txt  practical1  practical2  practical3  practical4  zombie  zombie1
output.txt practical1.c practical2.c practical3.c practical4.c zombie.c  zombie1.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical5a.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 2 — Paste the complete C program



The screenshot shows a terminal window with the title bar "monis@DESKTOP-C3GM4T6: ~/OSSP-Practical". The window contains the GNU nano 8.7.1 editor editing a file named "practical5a.c". The code in the editor is a C program that defines a constant, includes standard headers, and implements two functions: `elapsed_seconds` and `read_all`. The `elapsed_seconds` function calculates the time difference between two `timespec` structures. The `read_all` function reads data from a file descriptor into a buffer until the requested size is reached or an error occurs. The bottom of the window shows the nano editor's command palette with various shortcuts like `^G` for Help, `^O` for Write Out, `^F` for Where Is, `^K` for Cut, `^T` for Execute, `^C` for Location, `M-U` for Undo, `^X` for Exit, `^R` for Read File, `^F` for Replace, `^U` for Paste, `^J` for Justify, `^/_` for Go To Line, and `M-E` for Redo.

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical5a.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <errno.h>

#define COUNT 100000

static double elapsed_seconds(struct timespec start,
                             struct timespec end)
{
    return (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;
}

/* Read exactly the requested number of bytes. */
static int read_all(int fd, void *buffer, size_t size)
{
    size_t received = 0;
    char *ptr = buffer;

    while (received < size) {
        ssize_t n = read(fd, ptr + received, size - received);

        if (n == 0)
            return 0;

        if (n == -1) {
            if (errno == EINTR)
                continue;

            perror("read");
            return -1;
        }

        received += (size_t)n;
    }

    return 1;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify   ^/_ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical5a.c *
}

/* Write exactly the requested number of bytes. */
static int write_all(int fd, const void *buffer, size_t size)
{
    size_t sent = 0;
    const char *ptr = buffer;

    while (sent < size) {
        ssize_t n = write(fd, ptr + sent, size - sent);

        if (n == -1) {
            if (errno == EINTR)
                continue;

            perror("write");
            return -1;
        }

        if (n == 0)
            return -1;

        sent += (size_t)n;
    }

    return 0;
}

int main(void)
{
    int fd[2];

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    struct timespec start, end;

    if (clock_gettime(CLOCK_MONOTONIC, &start) == -1) {
        perror("clock_gettime");
        return 1;
    }

    pid_t pid = fork();

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    close(fd[0]);
    close(fd[1]);
    return 1;
}

if (pid == 0) {
    /* Child: consumer */
    close(fd[1]);

    int value;
    int received = 0;
    long long sum = 0;

    while (received < COUNT) {
        int result = read_all(fd[0], &value, sizeof(value));

        if (result == 0)
            break;

        if (result == -1) {
            close(fd[0]);
            _exit(1);
        }

        sum += value;
        received++;
    }

    close(fd[0]);

    printf("CHILD PROCESS - CONSUMER\n");
    printf("Child PID: %ld\n", (long)getpid());
    printf("Child consumed %d integers.\n", received);
    printf("Sum of received data: %lld\n", sum);

    fflush(stdout);

    _exit(received == COUNT ? 0 : 1);
}

/* Parent: producer */
```

```
/* Parent: producer */
close(fd[0]);

int sent = 0;
int write_failed = 0;

for (int i = 1; i <= COUNT; i++) {
    if (write_all(fd[1], &i, sizeof(i)) == -1) {
        write_failed = 1;
        break;
    }

    sent++;
}

close(fd[1]);

int status;

if (waitpid(pid, &status, 0) == -1) {
    perror("waitpid");
    return 1;
}

if (clock_gettime(CLOCK_MONOTONIC, &end) == -1) {
    perror("clock_gettime");
    return 1;
}

double seconds = elapsed_seconds(start, end);
double bytes = (double)sent * sizeof(int);

printf("\nPARENT PROCESS - PRODUCER\n");
printf("Parent PID: %ld\n", (long)getpid());
printf("Parent produced %d integers.\n", sent);
printf("Data transferred: %.0f bytes\n", bytes);
printf("Communication time: %.6f seconds\n", seconds);

printf("Throughput: %.2f MB/s\n",
       seconds > 0 ? bytes / seconds / 1000000.0 : 0.0);

if (write_failed ||
    !WIFEXITED(status) ||
    WEXITSTATUS(status) != 0) {
    fprintf(stderr, "Communication failed.\n");
}
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical5a.c *
    write_failed = 1;
    break;
}

    sent++;
}

close(fd[1]);

int status;

if (waitpid(pid, &status, 0) == -1) {
    perror("waitpid");
    return 1;
}

if (clock_gettime(CLOCK_MONOTONIC, &end) == -1) {
    perror("clock_gettime");
    return 1;
}

double seconds = elapsed_seconds(start, end);
double bytes = (double)sent * sizeof(int);

printf("\nPARENT PROCESS - PRODUCER\n");
printf("Parent PID: %ld\n", (long)getpid());
printf("Parent produced %d integers.\n", sent);
printf("Data transferred: %.0f bytes\n", bytes);
printf("Communication time: %.6f seconds\n", seconds);

printf("Throughput: %.2f MB/s\n",
       seconds > 0 ? bytes / seconds / 1000000.0 : 0.0);

if (write_failed ||
    !WIFEXITED(status) ||
    WEXITSTATUS(status) != 0) {
    fprintf(stderr, "Communication failed.\n");
    return 1;
}

printf("Communication completed successfully.\n");

return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

Save with Ctrl + O, Enter, then Ctrl + X.

```
Write to File: practical5a.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel    M-M Mac Format  M-P Prepend    ^Q Discard buffer
```

Step 3 — Compile and execute

```
gcc -Wall -Wextra practical5a.c -o practical5a
```

```
./practical5a
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical5a.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra practical5a.c -o practical5a
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical5a
CHILD PROCESS - CONSUMER
Child PID: 950
Child consumed 100000 integers.
Sum of received data: 5000050000

PARENT PROCESS - PRODUCER
Parent PID: 949
Parent produced 100000 integers.
Data transferred: 400000 bytes
Communication time: 0.044903 seconds
Throughput: 8.91 MB/s
Communication completed successfully.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 4 — Create practical5b.c

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical5b.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical5b.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    int fd[2];

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t pid1 = fork();

    if (pid1 < 0) {
        perror("fork");
        close(fd[0]);
        close(fd[1]);
        return 1;
    }

    if (pid1 == 0) {
        /* First child: execute ls -l */

        close(fd[0]);

        if (dup2(fd[1], STDOUT_FILENO) == -1) {
            perror("dup2");
            _exit(1);
        }

        close(fd[1]);

        execlp("ls", "ls", "-l", (char *)NULL);

        perror("execlp ls");
        _exit(127);
    }

    pid_t pid2 = fork();

    if (pid2 < 0) {

```

Help	Write Out	Where Is	Cut	Execute	Location	Undo
Exit	Read File	Replace	Paste	Justify	Go To Line	Redo

```
if (pid2 < 0) {
    perror("fork");

    close(fd[0]);
    close(fd[1]);

    waitpid(pid1, NULL, 0);

    return 1;
}

if (pid2 == 0) {
    /* Second child: execute grep ".c" */

    close(fd[1]);

    if (dup2(fd[0], STDIN_FILENO) == -1) {
        perror("dup2");
        _exit(1);
    }

    close(fd[0]);

    execlp("grep", "grep", ".c", (char *)NULL);

    perror("execlp grep");
    _exit(127);
}

/* Parent closes the pipe and waits. */

close(fd[0]);
close(fd[1]);

int status1;
int status2;

if (waitpid(pid1, &status1, 0) == -1) {
    perror("waitpid");
    return 1;
}

if (waitpid(pid2, &status2, 0) == -1) {
    perror("waitpid");
    return 1;
}
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical5b.c *
    perror("dup2");
    _exit(1);
}

close(fd[0]);

execlp("grep", "grep", ".c", (char *)NULL);

perror("execlp grep");
_exit(127);
}

/* Parent closes the pipe and waits. */

close(fd[0]);
close(fd[1]);

int status1;
int status2;

if (waitpid(pid1, &status1, 0) == -1) {
    perror("waitpid");
    return 1;
}

if (waitpid(pid2, &status2, 0) == -1) {
    perror("waitpid");
    return 1;
}

printf("\nPipeline execution completed.\n");

if (WIFEXITED(status1) &&
    WIFEXITED(status2) &&
    WEXITSTATUS(status1) == 0 &&
    WEXITSTATUS(status2) == 0) {
    printf("Both commands executed successfully.\n");
    return 0;
}

printf("One or more commands returned a nonzero status.\n");

return 1;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  ^U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line ^E Redo
```

Save using Ctrl + O, Enter, then Ctrl + X.

```
}
Write to File: practical5b.c
^G Help      M-D DOS Format M-A Append   M-B Backup File ^T Browse
^C Cancel    M-M Mac Format M-P Prepend  ^O Discard buffer
```

Step 5 — Compile and execute

```
gcc -Wall -Wextra practical5b.c -o practical5b
```

```
./practical5b
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical5b.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra practical5b.c -o practical5b
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical5b
-rwxr-xr-x 1 monis monis 16312 Aug  7 15:19 practical1
-rw-r--r-- 1 monis monis  442 Aug  7 15:12 practical1.c
-rwxr-xr-x 1 monis monis 16176 Aug  7 16:39 practical2
-rw-r--r-- 1 monis monis  440 Aug  7 16:39 practical2.c
-rwxr-xr-x 1 monis monis 16208 Aug 15 20:39 practical3
-rw-r--r-- 1 monis monis  963 Aug 15 20:39 practical3.c
-rwxr-xr-x 1 monis monis 16256 Aug 15 21:19 practical4
-rw-r--r-- 1 monis monis  1152 Aug 15 21:19 practical4.c
-rwxr-xr-x 1 monis monis 16792 Sep 28 19:27 practical5a
-rw-r--r-- 1 monis monis  3746 Sep 28 19:27 practical5a.c
-rwxr-xr-x 1 monis monis 16344 Sep 28 19:31 practical5b
-rw-r--r-- 1 monis monis  1840 Sep 28 19:30 practical5b.c
-rw-r--r-- 1 monis monis  572 Aug 15 21:28 zombie.c
-rw-r--r-- 1 monis monis  724 Aug 15 21:48 zombie1.c

Pipeline execution completed.
Both commands executed successfully.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 6 — Verify the pipeline

Execute the equivalent shell command:

```
ls -l | grep ".c"
```

Then run your program again:

```
./practical5b
```

The matching file listings should agree.

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ls -l | grep ".c"
-rwxr-xr-x 1 monis monis 16312 Aug  7 15:19 practical1
-rw-r--r-- 1 monis monis  442 Aug  7 15:12 practical1.c
-rwxr-xr-x 1 monis monis 16176 Aug  7 16:39 practical2
-rw-r--r-- 1 monis monis  440 Aug  7 16:39 practical2.c
-rwxr-xr-x 1 monis monis 16208 Aug 15 20:39 practical3
-rw-r--r-- 1 monis monis  963 Aug 15 20:39 practical3.c
-rwxr-xr-x 1 monis monis 16256 Aug 15 21:19 practical4
-rw-r--r-- 1 monis monis 1152 Aug 15 21:19 practical4.c
-rwxr-xr-x 1 monis monis 16792 Sep 28 19:27 practical5a
-rw-r--r-- 1 monis monis  3746 Sep 28 19:27 practical5a.c
-rwxr-xr-x 1 monis monis 16344 Sep 28 19:31 practical5b
-rw-r--r-- 1 monis monis 1840 Sep 28 19:30 practical5b.c
-rw-r--r-- 1 monis monis  572 Aug 15 21:28 zombie.c
-rw-r--r-- 1 monis monis  724 Aug 15 21:48 zombie1.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical5b
-rwxr-xr-x 1 monis monis 16312 Aug  7 15:19 practical1
-rw-r--r-- 1 monis monis  442 Aug  7 15:12 practical1.c
-rwxr-xr-x 1 monis monis 16176 Aug  7 16:39 practical2
-rw-r--r-- 1 monis monis  440 Aug  7 16:39 practical2.c
-rwxr-xr-x 1 monis monis 16208 Aug 15 20:39 practical3
-rw-r--r-- 1 monis monis  963 Aug 15 20:39 practical3.c
-rwxr-xr-x 1 monis monis 16256 Aug 15 21:19 practical4
-rw-r--r-- 1 monis monis 1152 Aug 15 21:19 practical4.c
-rwxr-xr-x 1 monis monis 16792 Sep 28 19:27 practical5a
-rw-r--r-- 1 monis monis  3746 Sep 28 19:27 practical5a.c
-rwxr-xr-x 1 monis monis 16344 Sep 28 19:31 practical5b
-rw-r--r-- 1 monis monis 1840 Sep 28 19:30 practical5b.c
-rw-r--r-- 1 monis monis  572 Aug 15 21:28 zombie.c
-rw-r--r-- 1 monis monis  724 Aug 15 21:48 zombie1.c

Pipeline execution completed.
Both commands executed successfully.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Final verification

Run:

```
cd ~/OSSP-Practical
```

```
ls -l practical5a.c practical5b.c
```

```
ls -l practical5a practical5b
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ls -l practical5a.c practical5b.c
-rw-r--r-- 1 monis monis 3746 Sep 28 19:27 practical5a.c
-rw-r--r-- 1 monis monis 1840 Sep 28 19:30 practical5b.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ls -l practical5a practical5b
-rwxr-xr-x 1 monis monis 16792 Sep 28 19:27 practical5a
-rwxr-xr-x 1 monis monis 16344 Sep 28 19:31 practical5b
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

6. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Step 1 — Create the server

cd ~/OSSP-Practical

nano practical6a_server.c

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical6a_server.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical6a_server.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <errno.h>
#include <ctype.h>

#define SERVER_FIFO "/tmp/ossq6_server"
#define MAX_MSG 200

typedef struct {
    pid_t pid;
    char message[MAX_MSG];
} Request;

int main(void)
{
    Request req;
    char response_fifo[100];
    char response[MAX_MSG];

    if (mkfifo(SERVER_FIFO, 0666) == -1 && errno != EEXIST) {
        perror("mkfifo");
        return 1;
    }

    printf("FIFO server started.\n");
    printf("Waiting for clients...\n");
    fflush(stdout);

    int server_fd = open(SERVER_FIFO, O_RDWR);

    if (server_fd == -1) {
        perror("open");
        return 1;
    }

    while (1) {
        ssize_t n = read(server_fd, &req, sizeof(req));

        if (n == -1) {
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical6a_server.c *
ssize_t n = read(server_fd, &req, sizeof(req));

if (n == -1) {
    if (errno == EINTR)
        continue;
    perror("read");
    break;
}

if (n != sizeof(req))
    continue;

printf("Received from client %ld: %s\n",
       (long)req.pid, req.message);
fflush(stdout);

snprintf(response, sizeof(response), "%s", req.message);

for (int i = 0; response[i]; i++) {
    response[i] =
        (char)toupper((unsigned char)response[i]);
}

snprintf(response_fifo, sizeof(response_fifo),
         "/tmp/ossq6_client_%ld", (long)req.pid);

int client_fd = open(response_fifo, O_WRONLY);

if (client_fd == -1) {
    perror("open client FIFO");
    continue;
}

if (write(client_fd, response, strlen(response) + 1) == -1)
    perror("write");

close(client_fd);
}

close(server_fd);
unlink(SERVER_FIFO);

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Save with Ctrl + O, Enter, then Ctrl + X.

```
Write to File: practical6a_server.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse  
^C Cancel    M-M Mac Format  M-P Prepend    ^O Discard buffer
```

Step 2 — Create the client
nano practical6a_client.c

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical6a_client.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <errno.h>

#define SERVER_FIFO "/tmp/ossq6_server"
#define MAX_MSG 200

typedef struct {
    pid_t pid;
    char message[MAX_MSG];
} Request;

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "Usage: %s \"message\\n\\n\", argv[0]);
        return 1;
    }

    Request req = {0};
    req.pid = getpid();
    snprintf(req.message, sizeof(req.message), "%s", argv[1]);

    char response_fifo[100];

    snprintf(response_fifo, sizeof(response_fifo),
             "/tmp/ossq6_client_%ld", (long)req.pid);

    if (mkfifo(response_fifo, 0600) == -1) {
        perror("mkfifo");
        return 1;
    }

    int server_fd = open(SERVER_FIFO, O_WRONLY);

    if (server_fd == -1) {
        perror("Server not running");
        unlink(response_fifo);
        return 1;
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical6a_client.c *
    perror("mkfifo");
    return 1;
}

int server_fd = open(SERVER_FIFO, O_WRONLY);

if (server_fd == -1) {
    perror("Server not running");
    unlink(response_fifo);
    return 1;
}

if (write(server_fd, &req, sizeof(req)) != sizeof(req)) {
    perror("write");
    close(server_fd);
    unlink(response_fifo);
    return 1;
}

close(server_fd);

int client_fd = open(response_fifo, O_RDONLY);

if (client_fd == -1) {
    perror("open");
    unlink(response_fifo);
    return 1;
}

char response[MAX_MSG];
ssize_t n = read(client_fd, response, sizeof(response) - 1);

if (n > 0) {
    response[n] = '\0';
    printf("Server response: %s\n", response);
} else {
    printf("No response received.\n");
}

close(client_fd);
unlink(response_fifo);

return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo

Write to File: practical6a_client.c
^G Help      M-D DOS Format M-A Append   M-B Backup File ^T Browse
^C Cancel    M-M Mac Format M-P Prepend  ^Q Discard buffer
```

Step 3 — Compile both programs

gcc -Wall -Wextra practical6a_server.c -o practical6a_server

gcc -Wall -Wextra practical6a_client.c -o practical6a_client

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical6a_client.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra practical6a_server.c -o practical6a_server
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra practical6a_client.c -o practical6a_client
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 4 — Run the server and client

Open two Ubuntu terminals.

Terminal 1 — Server

```
cd ~/OSSP-Practical
./practical6a_server
```

Terminal 2 — Client

```
cd ~/OSSP-Practical
./practical6a_client "hello ossp"
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical6a_client.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra practical6a_server.c -o practical6a_server
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra practical6a_client.c -o practical6a_client
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical6a_server
FIFO server started.
Waiting for clients...
Received from client 1450: hello ossp

monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical6a_client "hello ossp"
Server response: HELLO OSSP
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 5 — Test multiple clients

Keep the server running in Terminal 1. In Terminal 2, run:

```
./practical6a_client "first client" &
./practical6a_client "second client" &
./practical6a_client "third client" &
wait
```

```
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra practical6a_server.c -o practical6a_server
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra practical6a_client.c -o practical6a_client
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical6a_server
FIFO server started.
Waiting for clients...
Received from client 1450: hello ossp
^C
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical6a_client "first client" &
./practical6a_client "second client" &
./practical6a_client "third client" &
wait
[1] 1473
[2] 1479
[3] 1485

monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical6a_client "hello ossp"
Server response: HELLO OSSP
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical6a_client "first client" &
./practical6a_client "second client" &
./practical6a_client "third client" &
wait
[1] 1501
[2] 1507
[3] 1513
```

Step 1 — Restart the server

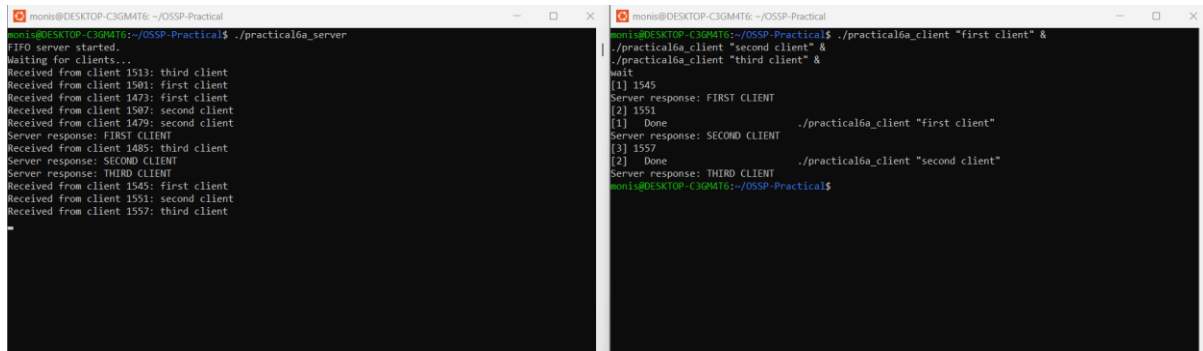
In the left terminal, press Ctrl+C if necessary to stop the waiting client commands, then run:

```
./practical6a_server
```

Step 2 — Test three clients

In the right terminal, first press Ctrl+C if the previous wait command is still blocking. Then run:

```
./practical6a_client "first client" &  
./practical6a_client "second client" &  
./practical6a_client "third client" &  
wait
```



The image shows two terminal windows side-by-side. The left window is titled 'monis@DESKTOP-C3GM4T6: ~/OSSP-Practical' and shows the output of running './practical6a_server'. The output indicates the server is started, waiting for clients, and then receives connections from three clients (1513, 1501, 1473) and responds to them. The right window is also titled 'monis@DESKTOP-C3GM4T6: ~/OSSP-Practical' and shows the output of running './practical6a_client "first client" &', './practical6a_client "second client" &', './practical6a_client "third client" &', and 'wait'. The output shows the clients sending data to the server and the server responding with 'FIRST CLIENT', 'SECOND CLIENT', and 'THIRD CLIENT' respectively.

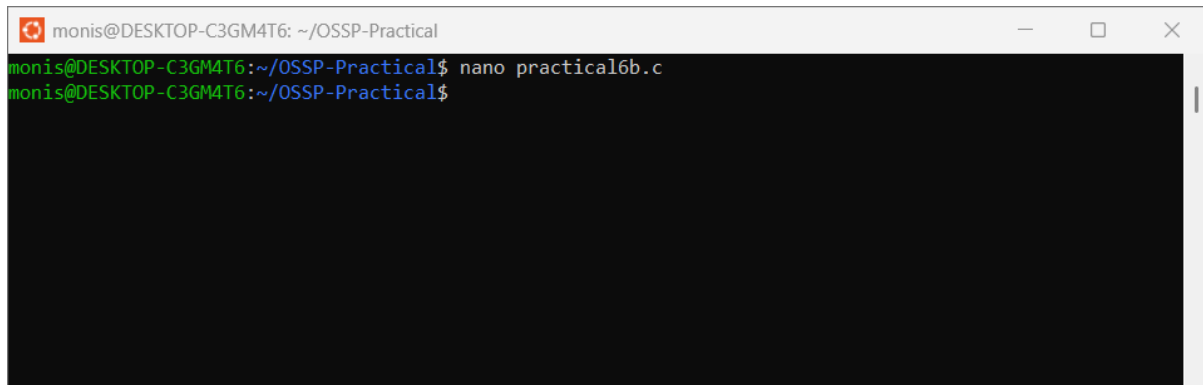
```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical  
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical6a_server  
PIFO server started.  
Waiting for clients...  
Received from client 1513: third client  
Received from client 1501: first client  
Received from client 1473: first client  
Received from client 1507: second client  
Received from client 1479: second client  
Server response: FIRST CLIENT  
Received from client 1485: third client  
Server response: SECOND CLIENT  
Server response: THIRD CLIENT  
Received from client 1545: first client  
Received from client 1551: second client  
Received from client 1557: third client  
-  
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical6a_client "first client" &  
./practical6a_client "second client" &  
./practical6a_client "third client" &  
wait  
[1] 1545  
Server response: FIRST CLIENT  
[2] 1551  
[1] Done  
Server response: SECOND CLIENT  
[3] 1557  
[2] Done  
Server response: THIRD CLIENT  
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

--POSIX Signal Handling

Step 6 — Create practical6b.c

cd ~/OSSP-Practical

nano practical6b.c



The image shows a terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSSP-Practical'. The user has entered the command 'nano practical6b.c' and the prompt has changed to 'monis@DESKTOP-C3GM4T6:~/OSSP-Practical\$'.

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical  
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical6b.c  
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical6b.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

volatile sig_atomic_t sigint_received = 0;
volatile sig_atomic_t sigterm_received = 0;
volatile sig_atomic_t sigusr1_received = 0;

void signal_handler(int signo)
{
    if (signo == SIGINT)
        sigint_received = 1;
    else if (signo == SIGTERM)
        sigterm_received = 1;
    else if (signo == SIGUSR1)
        sigusr1_received = 1;
}

int main(void)
{
    struct sigaction sa = {0};

    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);

    if (sigaction(SIGINT, &sa, NULL) == -1 ||
        sigaction(SIGTERM, &sa, NULL) == -1 ||
        sigaction(SIGUSR1, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("Signal handling program started.\n");
    printf("Process PID: %ld\n", (long)getpid());
    printf("Press Ctrl+C or send SIGUSR1/SIGTERM.\n");
    fflush(stdout);

    while (!sigterm_received) {
        pause();

        if (sigint_received) {
            sigint_received = 0;
        }
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical6b.c *
    else if (signo == SIGUSR1)
        sigusr1_received = 1;
}

int main(void)
{
    struct sigaction sa = {0};

    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);

    if (sigaction(SIGINT, &sa, NULL) == -1 ||
        sigaction(SIGTERM, &sa, NULL) == -1 ||
        sigaction(SIGUSR1, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("Signal handling program started.\n");
    printf("Process PID: %ld\n", (long) getpid());
    printf("Press Ctrl+C or send SIGUSR1/SIGTERM.\n");
    fflush(stdout);

    while (!sigterm_received) {
        pause();

        if (sigint_received) {
            sigint_received = 0;
            printf("SIGINT received: Ctrl+C detected.\n");
        }

        if (sigusr1_received) {
            sigusr1_received = 0;
            printf("SIGUSR1 received: User-defined signal.\n");
        }

        fflush(stdout);
    }

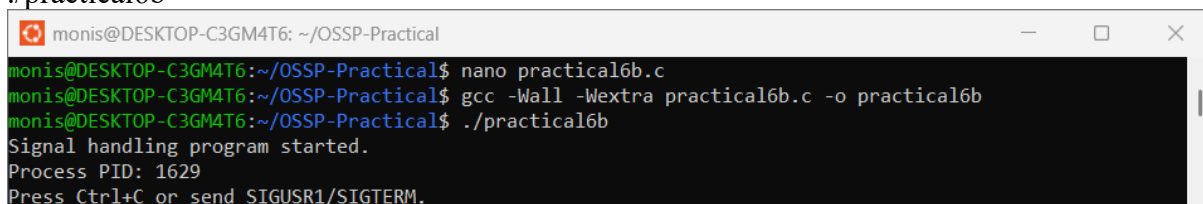
    printf("SIGTERM received: Terminating gracefully.\n");
    printf("Program exited successfully.\n");

    return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^Y Exit      ^R Read File ^N Replace   ^I Paste     ^J Justify   ^V Go To Line M-F Redo
Write to File: practical6b.c
^G Help      M-D DOS Format  M-A Append   M-B Backup File ^T Browse
^C Cancel    M-M Mac Format  M-P Prepend  ^O Discard buffer
```

Step 7 — Compile and run

```
gcc -Wall -Wextra practical6b.c -o practical6b
./practical6b
```



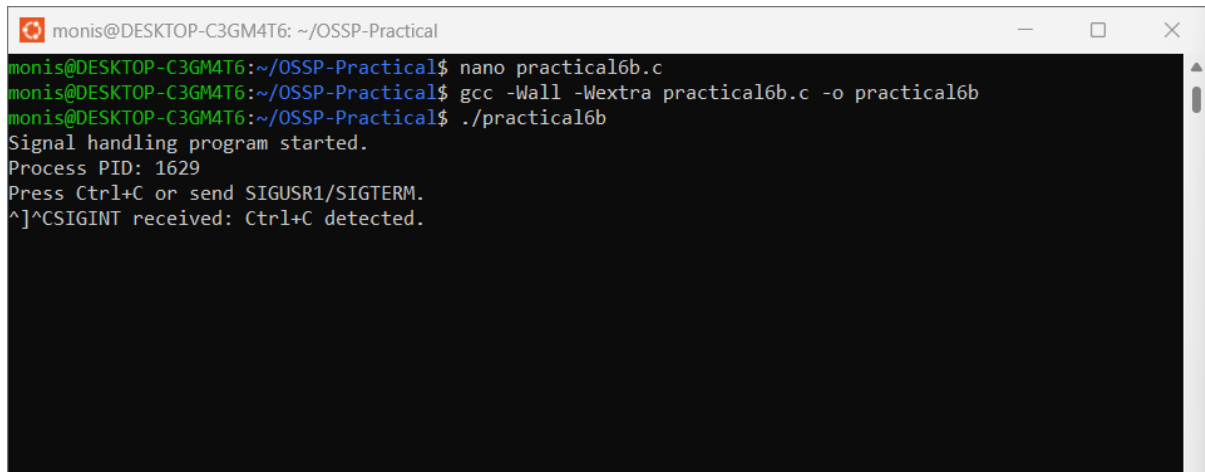
```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical$ nano practical6b.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra practical6b.c -o practical6b
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical6b
Signal handling program started.
Process PID: 1629
Press Ctrl+C or send SIGUSR1/SIGTERM.
```

Step 8 — Test all three signals

Leave the program running in Terminal 1.

In Terminal 1, press Ctrl + C to test SIGINT. The program should display:

SIGINT received: Ctrl+C detected.



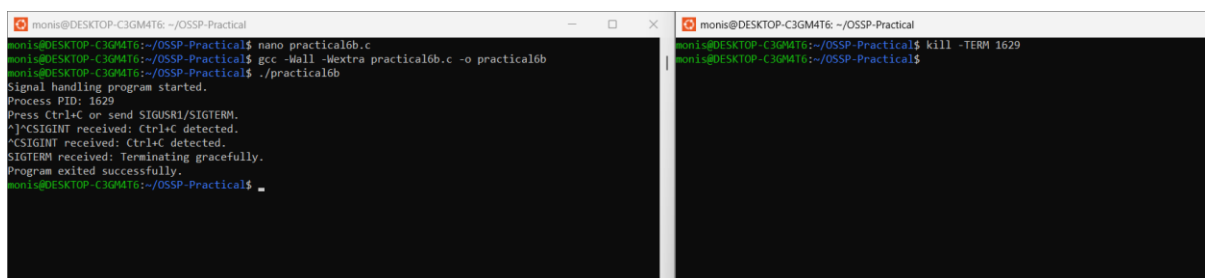
```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical6b.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra practical6b.c -o practical6b
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical6b
Signal handling program started.
Process PID: 1629
Press Ctrl+C or send SIGUSR1/SIGTERM.
^J^CSIGINT received: Ctrl+C detected.
```

Open Terminal 2 and send SIGUSR1 using the actual PID displayed by your program:

kill -USR1 1629

Finally, send SIGTERM:

kill -TERM 1629



```
monis@DESKTOP-C3GM4T6:~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical6b.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra practical6b.c -o practical6b
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical6b
Signal handling program started.
Process PID: 1629
Press Ctrl+C or send SIGUSR1/SIGTERM.
^J^CSIGINT received: Ctrl+C detected.
^CSIGINT received: Ctrl+C detected.
SIGTERM received: Terminating gracefully.
Program exited successfully.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$

monis@DESKTOP-C3GM4T6:~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ kill -TERM 1629
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```